

AIQT Milestone 17 Build Specification

Incremental Work Graph Expansion and Roadmap Placeholder Replacement

AIQT Project

Version 0.2

Revision Notes - v0.2

1. Source Alignment
2. Milestone Objective
3. Problems to Solve
 - 3.1 One-shot planning blocks progressive delivery
 - 3.2 Manual canonical-state editing is unsafe
 - 3.3 New projects must retain one continuous AIQT history
 - 3.4 Placeholder replacement must not release future work prematurely
4. Commands in Scope
 - 4.1 File-input extension
 - 4.2 Stdin/import extension
 - 4.3 Prompt support
 - 4.4 Existing behavior preserved
5. Out of Scope
6. Behavioral Position in the Workflow
7. Extension Input Contract
 - 7.1 Required extension fields
 - 7.2 Entry work units
 - 7.3 Exit work units
 - 7.4 Client-key uniqueness
8. Placeholder Eligibility Contract
 - 8.1 Eligible statuses
 - 8.2 Ineligible statuses
 - 8.3 Additional eligibility rules
 - 8.4 Placeholder mutation
9. Dependency Preservation and Rewiring Rules
 - 9.1 Incoming dependencies
 - 9.2 Outgoing dependencies
 - 9.3 Non-blocking relationships
 - 9.4 No premature readiness
 - 9.5 Full-graph validation
10. Candidate-State and Atomic Mutation Contract
 - 10.1 No partial writes
 - 10.2 Atomic persistence
 - 10.3 Preview
11. Canonical State and Schema Rules
 - 11.1 Project state
 - 11.2 Schema version
 - 11.3 Existing records
12. Runlog Contract
 - 12.1 Successful extension
 - 12.2 Preview and failure
13. Workflow Recalculation Contract

- 14. Command Output Contract
 - 14.1 Human-readable success output
 - 14.2 JSON output
- 15. Error Handling and Exit Codes
- 16. Prompt and Agent-Driver Updates
- 17. Repository Implementation Scope
- 18. Runtime and Package Impact
- 19. Testing Scope
 - 19.1 Unit tests
 - 19.2 Integration tests
 - 19.3 Atomicity tests
 - 19.4 Finançs dogfood regression fixture
 - 19.5 Mixed boundary-dependency fixture
 - 19.6 Regression commands
- 20. Definition of Done
- 21. Source-Control and Release Contract
 - Branch
 - Primary commit
 - Milestone tag
- 22. Agent Execution Packet
 - Starting point
 - Required implementation
 - Explicitly out of scope
- Appendix A — Minimum Test Matrix
- Appendix B — Finançs Acceptance Scenario
- Appendix C — Compatibility Rules

Milestone: M17**Title:** Incremental Work Graph Expansion and Roadmap Placeholder Replacement**Status:** Build-ready**Depends on:** M1-M16**Starting baseline:** `main` at merge commit `d5dcb395f3ca56d36b38540960239403eef09f25`**Starting tag:** `m16-project-init-ux-root-configuration`**Expected baseline:** 878/878 tests passing**Primary dogfood:** `financas-pessoais`, after Corte 0, with WU001-WU014 completed and WU015 ready as the Corte 1 roadmap placeholder.

Revision Notes - v0.2

This revision resolves the two correctness findings raised during review and clarifies two compatibility details before build handoff:

- preserves the established exit-code taxonomy: workflow-position gates return exit `2`, while invalid input or invalid canonical structure returns exit `3`;
- confirms ordinary `PLAN-GRAPH-NOT-EMPTY` behavior remains exit `2`;
- treats extension against an empty graph as a blocked workflow-position gate with exit `2` and recommends ordinary `aiqt plan`;
- preserves both blocking dependency types, `blocks` and `requires`, during placeholder-boundary rewiring;
- preserves the original blocking dependency type instead of converting `requires` to `blocks`;
- keeps `relates_to` non-blocking and non-propagated by default;
- keeps the command action compatible as `action: "plan"`, with extension mode represented in command data;
- explicitly defines `--extend` without `--replace-placeholder` as invalid CLI input with exit `3`;
- expands the minimum matrix from 42 to 48 scenarios, including a mixed boundary-dependency fixture.

1. Source Alignment

M17 is a corrective milestone derived from a real multi-cut project dogfood.

AIQT currently supports:

Capture → Plan → Handoff → Execute → Checkpoint → Review → Continue

However, the current planning engine only permits the first plan operation. Once a work graph exists, both `aiqt plan` and `aiqt import plan` reject new plan input with:

`PLAN-GRAPH-NOT-EMPTY`
A work graph already exists. `aiqt plan` does not replan an existing graph.

That behavior protects an existing graph from accidental replacement, but it prevents the intended progressive planning model:

```

Scope total
→ plan current executable cut
→ execute cut
→ review checkpoint
→ detail next cut
→ extend the same canonical graph

```

The Finanças dogfood confirmed the gap:

- Corte 0 completed through 14 bounded work units;
- the existing canonical state and runlog must remain authoritative;
- WU015 is a ready roadmap placeholder for Corte 1;
- the agent cannot replace that placeholder with detailed milestones and work units using any supported AIQT command;
- direct `.aiqt/state.json` editing would bypass AIQT mutation, validation, and runlog contracts;
- creating a second AIQT project would fragment project history.

M17 must add safe incremental graph extension without weakening the original M3 protection against accidental replanning.

2. Milestone Objective

M17 adds an official, transactional method to extend an existing work graph and replace one not-started roadmap placeholder with a detailed execution subgraph.

The target workflow is:

```

Completed historical graph
→ roadmap placeholder ready/planned/blocked
→ preview extension
→ validate complete candidate graph
→ mark placeholder replanned
→ add detailed milestones/work units/dependencies
→ preserve historical state and runlog
→ continue with aiqt next

```

M17 must prove that AIQT can:

1. preserve all completed milestones, work units, checkpoints, packets, dependencies, acknowledgments, decisions, and runlog history;
2. add new milestones, work units, and dependencies to a non-empty graph;
3. replace one eligible placeholder work unit without deleting it;
4. use the existing `replanned` work-unit terminal status rather than inventing a new `super-seded` status;
5. preserve original dependency records and add the new dependencies required to maintain execution order;
6. preview the complete mutation without writing canonical files or runlog events;
7. reject invalid extensions before any partial write;
8. continue normal `status`, `review`, `manage`, `next`, `checkpoint`, `export`, and `repair` behavior after extension;
9. keep ordinary `aiqt plan` behavior unchanged when `--extend` is absent.

3. Problems to Solve

3.1 One-shot planning blocks progressive delivery

Large projects should not require every future milestone and work unit to be implementation-ready at initial capture time. AIQT currently forces one of two poor outcomes:

- over-plan the full project before implementation evidence exists; or
- insert roadmap placeholders that cannot later be expanded through supported commands.

3.2 Manual canonical-state editing is unsafe

Editing `.aiqt/state.json` directly can bypass:

- schema validation;
- ID generation;
- dependency and cycle validation;
- readiness recalculation;
- current workflow position recalculation;
- atomic writes;
- append-only runlog history;
- compatibility logic for exports and reviews.

M17 must eliminate the need for this workaround.

3.3 New projects must retain one continuous AIQT history

A new AIQT project per cut would split:

- canonical state;
- decisions;
- requirements;
- checkpoints;
- review findings;
- work-unit identity;
- project exports;
- manager classification.

M17 must keep every cut in the same `.aiqt/` project.

3.4 Placeholder replacement must not release future work prematurely

When WU015 currently blocks a future roadmap placeholder, marking WU015 `replanned` alone may make the old dependency appear satisfied. The replacement subgraph must add blocking dependencies from its exit work unit or units to the original downstream dependents.

4. Commands in Scope

4.1 File-input extension

```
aiqt plan --extend --replace-placeholder WU015 --from-file <path>
aiqt plan --extend --replace-placeholder WU015 --from-file <path> --preview
aiqt plan --extend --replace-placeholder WU015 --from-file <path> --json
aiqt plan --extend --replace-placeholder WU015 --from-file <path> --preview --json
```

4.2 Stdin/import extension

```
aiqt import plan --stdin --extend --replace-placeholder WU015
aiqt import plan --stdin --extend --replace-placeholder WU015 --preview
aiqt import plan --stdin --extend --replace-placeholder WU015 --json
aiqt import plan --stdin --extend --replace-placeholder WU015 --preview --json
```

`aiqt import plan --from-file` must support the same extension options if that transport already delegates through the import command.

4.3 Prompt support

```
aiqt prompt plan --extend --replace-placeholder WU015
aiqt prompt plan --extend --replace-placeholder WU015 --json
```

The prompt must explain how to produce a bounded extension payload for the named placeholder. Prompt generation remains read-only.

4.4 Existing behavior preserved

```
aiqt plan --from-file <path>
aiqt import plan --stdin
```

When a graph already exists and `--extend` is not supplied, these commands must continue to return `PLAN-GRAPH-NOT-EMPTY`.

5. Out of Scope

M17 must not add:

- arbitrary editing of existing milestones or work units;
- deletion of historical milestones, work units, dependencies, checkpoints, packets, issues, or runlog events;
- replacement of more than one placeholder in a single operation;
- replacement of an `in_progress`, `needs_review`, `done`, `replanned`, or `cancelled` work unit;
- automatic AI generation of the extension plan;
- autonomous implementation of the newly created work units;
- checkpoint amendment;
- historical packet-body reconstruction;
- arbitrary dependency removal;
- milestone renaming or scope rewriting for completed milestones;
- cross-project graph merge;

- graph compaction;
- schema redesign unrelated to extension;
- package publishing or release automation;
- manual mutation of consumer projects.

6. Behavioral Position in the Workflow

M17 does not add a new workflow stage. It adds a valid mutation between cut review and the next handoff.

```
Capture
→ initial Plan
→ Execute current cut
→ Checkpoint
→ Review current cut
→ Extend Plan
→ Handoff next detailed work unit
→ Continue
```

Recommended operator sequence:

```
aiqt status
aiqt review
aiqt prompt plan --extend --replace-placeholder WU015
aiqt plan --extend --replace-placeholder WU015 --from-file corte-1-plan.json --preview
aiqt plan --extend --replace-placeholder WU015 --from-file corte-1-plan.json
aiqt graph validate
aiqt review
aiqt status
aiqt next --preview
```

7. Extension Input Contract

M17 introduces a plan-extension input contract that reuses the existing milestone, work-unit, and dependency plan schemas.

Conceptual shape:


```

{
  "extension": {
    "entryWorkUnitClientKeys": ["C1-WU01"],
    "exitWorkUnitClientKeys": ["C1-WU14"],
    "reason": "Expand the Corte 1 roadmap placeholder after Corte 0 completion."
  },
  "milestones": [
    {
      "clientKey": "C1-M1",
      "title": "Financial Domain Foundation",
      "objective": "...",
      "acceptanceCriteria": ["..."]
    }
  ],
  "workUnits": [
    {
      "clientKey": "C1-WU01",
      "milestoneClientKey": "C1-M1",
      "title": "Extend the financial schema",
      "objective": "...",
      "scope": ["..."],
      "outOfScope": ["..."],
      "acceptanceCriteria": ["..."],
      "validationCommands": ["..."]
    }
  ],
  "dependencies": [
    {
      "fromClientKey": "C1-WU01",
      "toClientKey": "C1-WU02",
      "type": "blocks"
    },
    {
      "fromClientKey": "C1-WU02",
      "toClientKey": "C1-WU03",
      "type": "requires"
    }
  ]
}

```

The exact field names must align with the current plan schema and command-result conventions. Do not create duplicate parallel representations when equivalent current fields already exist.

7.1 Required extension fields

The extension payload must identify:

- at least one new milestone;
- at least one new work unit;
- at least one entry work unit client key;
- at least one exit work unit client key;
- a non-empty reason.

7.2 Entry work units

Entry work units are the first executable nodes of the replacement subgraph.

Rules:

- every entry key must reference a work unit in the new payload;
- entry work units may also have internal dependencies, but must not depend on another new unit that prevents all declared entries from being actual graph entries;
- the engine copies every existing incoming blocking dependency of type `blocks` or `requires` to every declared entry work unit;
- each copied dependency preserves the original dependency type exactly;

- copied dependencies receive new canonical IDs.

7.3 Exit work units

Exit work units represent completion of the replacement subgraph.

Rules:

- every exit key must reference a work unit in the new payload;
- for every existing outgoing blocking dependency of type `blocks` or `requires`, the engine adds a new dependency from every declared exit work unit to the existing downstream work unit;
- each generated dependency preserves the original dependency type exactly;
- downstream work therefore cannot become ready merely because the placeholder becomes `replanned`;
- multiple exit work units use AND semantics through independent blocking dependencies.

7.4 Client-key uniqueness

New milestone and work-unit client keys must be unique across:

- the extension payload;
- existing canonical client keys;
- previous extension operations.

Canonical IDs continue from the current highest ID according to existing ID-generation rules.

8. Placeholder Eligibility Contract

The target named by `--replace-placeholder` must exist in canonical state.

8.1 Eligible statuses

Allowed:

- `planned`;
- `ready`;
- `blocked`, where `blocked` is represented by current project conventions rather than a new status if the engine derives blocking from dependencies.

8.2 Ineligible statuses

Rejected:

- `in_progress`;
- `needs_review`;
- `done`;
- `replanned`;
- `cancelled`.

8.3 Additional eligibility rules

The placeholder must:

- have no checkpoint;
- not be the current in-progress work unit;

- not have an active agent packet;
- not have implementation evidence recorded as completed work;
- not already have replacement metadata;
- belong to the current project graph.

8.4 Placeholder mutation

On successful extension:

- the placeholder remains in canonical state;
- its status becomes `replanned` using existing M11–M16 conventions;
- the replanning reason is recorded;
- replacement work-unit IDs are recorded using existing replanning metadata where possible;
- if current schema lacks replacement linkage, add the smallest additive optional field required, such as `replacedByWorkUnitIds`;
- the placeholder is never selected by `aiqt next` again;
- exports and manager output show it as replanned, not done.

M17 must not introduce a new `superseded` status.

9. Dependency Preservation and Rewiring Rules

M17 must preserve existing dependency records. It must not delete historical dependency objects as part of extension.

9.1 Incoming dependencies

For every existing blocking dependency of type `blocks` or `requires`:

existing predecessor → placeholder

add:

existing predecessor → each new entry work unit

Rules:

- preserve the original dependency type exactly;
- a `requires` dependency remains `requires`;
- a `blocks` dependency remains `blocks`;
- the original dependency remains for audit history.

9.2 Outgoing dependencies

For every existing blocking dependency of type `blocks` or `requires`:

placeholder → existing downstream work unit

add:

each new exit work unit → existing downstream work unit

Rules:

- preserve the original dependency type exactly;
- a `requires` dependency remains `requires`;
- a `blocks` dependency remains `blocks`;
- the original dependency remains for audit history.

9.3 Non-blocking relationships

Existing `relates_to` or equivalent non-blocking dependency types are not copied automatically and must not affect readiness. The extension payload may add explicit new non-blocking relationships.

9.4 No premature readiness

After extension:

- the first replacement entry becomes ready only when all copied incoming `blocks` and `requires` dependencies are satisfied;
- future roadmap placeholders remain blocked until every copied outgoing `blocks` and `requires` dependency is satisfied;
- a replanned placeholder never independently releases downstream work;
- mixed boundary types must produce the same readiness result as their original canonical semantics.

9.5 Full-graph validation

The candidate graph must be checked for:

- self-dependencies;
- duplicate dependencies;
- missing work-unit references;
- cycles across old and new nodes;
- impossible entry/exit mappings;
- invalid cross-milestone references;
- dependency readiness regressions.

10. Candidate-State and Atomic Mutation Contract

M17 must use a candidate-state transaction model.

```

Read canonical state
→ parse extension input
→ validate placeholder eligibility
→ allocate candidate IDs
→ build candidate milestones/work units/dependencies
→ mark placeholder replanned in candidate only
→ copy boundary dependencies
→ validate complete candidate schema
→ validate complete candidate graph
→ recompute statuses and workflow position
→ generate preview or atomically persist
→ append runlog only after successful persistence

```

10.1 No partial writes

If any validation or write step fails:

- `project.json` is unchanged;
- `state.json` is unchanged;
- `runlog.jsonl` is unchanged;
- exports are unchanged;
- no IDs are consumed canonically;
- no placeholder status changes occur.

10.2 Atomic persistence

Use the repository's existing atomic-write mechanism. Do not introduce a second state-writing path.

10.3 Preview

`--preview` performs all parsing, ID simulation, placeholder checks, boundary dependency generation, and full candidate validation, but:

- writes no canonical file;
- appends no runlog event;
- changes no status;
- changes no `currentMilestoneId` or `currentWorkUnitId`;
- creates no packet;
- creates no export.

11. Canonical State and Schema Rules

M17 preserves:

- `.aiqt/project.json` as canonical project context;
- `.aiqt/state.json` as canonical workflow state;
- `.aiqt/runlog.jsonl` as append-only workflow history;
- `.aiqt/exports/` as generated views only.

11.1 Project state

The extension operation should not mutate requirements, constraints, decisions, assumptions, risks, or quality definitions unless the existing plan command already has an explicit additive contract for them. M17 is a work-graph extension, not a context-update command.

11.2 Schema version

No `AIQT_SCHEMA_VERSION` bump is expected if existing `replanned` metadata is sufficient.

If replacement linkage requires an additive optional field:

- maintain backward compatibility;
- old projects must load unchanged;
- absent field means no recorded replacement list;
- migration must be automatic or unnecessary under current schema parsing conventions.

11.3 Existing records

Completed work units, checkpoints, issues, packets, milestones, and dependencies must remain semantically unchanged. Timestamps may only change on records explicitly mutated by the extension operation and on project-level derived metadata where current conventions require it.

12. Runlog Contract

12.1 Successful extension

Append deterministic events after canonical persistence succeeds.

Required semantic events:

- `plan.extended` ;
- `work_unit.replanned` for the replaced placeholder.

If current runlog conventions prefer one event with structured affected items, reuse those conventions rather than duplicating information.

The extension event must include at least:

- target placeholder work-unit ID;
- reason;
- added milestone IDs;
- added work-unit IDs;
- added dependency IDs;
- entry work-unit IDs;
- exit work-unit IDs;
- copied incoming dependency IDs;
- copied outgoing dependency IDs;
- resulting next recommended command.

12.2 Preview and failure

Do not append runlog events for:

- preview;
 - malformed input;
 - invalid schema;
 - ineligible placeholder;
 - duplicate keys;
 - cycle detection;
 - atomic-write failure;
 - any operation that leaves canonical state unchanged.
-

13. Workflow Recalculation Contract

After successful extension, AIQT must recompute using existing centralized services:

- milestone derived statuses;
- work-unit readiness;
- current milestone;

- current work unit;
- project status;
- next recommended command;
- review/manage classifications where derived dynamically.

Expected Finançs result:

```
WU001-WU014: done
WU015: replanned
First detailed Corte 1 entry WU: ready
Other detailed Corte 1 WUs: planned/blocked
Corte 2+ placeholders: planned/blocked
currentWorkUnitId: null
nextRecommendedCommand: aigt next --preview or the current canonical equivalent
```

M17 must not generate an agent packet automatically.

14. Command Output Contract

14.1 Human-readable success output

Must show:

- placeholder replaced;
- reason;
- milestones added;
- work units added;
- dependencies added;
- entry work units;
- exit work units;
- copied boundary dependencies;
- completed work units modified: 0;
- completed milestones modified: 0;
- cycles introduced: 0;
- mutation performed: yes/no;
- next recommended command.

14.2 JSON output

Conceptual result:

```

{
  "status": "passed",
  "action": "plan",
  "projectStatus": "in_progress",
  "summary": "Extended the work graph and replanned placeholder WU015.",
  "completedActions": [
    "Validated extension input",
    "Validated placeholder eligibility",
    "Validated candidate graph",
    "Added milestones, work units, and dependencies",
    "Marked WU015 replanned"
  ],
  "changedFiles": [
    ".aiqt/state.json",
    ".aiqt/runlog.jsonl"
  ],
  "affectedItems": ["WU015", "M013", "WU022"],
  "blockingIssues": [],
  "warnings": [],
  "requiresHumanInput": false,
  "nextRecommendedCommand": "aiqt next --preview",
  "exitCode": 0,
  "data": {
    "operation": "extend",
    "preview": false,
    "placeholderWorkUnitId": "WU015",
    "placeholderFinalStatus": "replanned",
    "addedMilestoneIds": ["M013"],
    "addedWorkUnitIds": ["WU022"],
    "addedDependencyIds": ["DEP-029"],
    "entryWorkUnitIds": ["WU022"],
    "exitWorkUnitIds": ["WU035"],
    "completedWorkUnitsModified": 0,
    "completedMilestonesModified": 0,
    "cyclesIntroduced": 0
  }
}

```

The action string must remain compatible with the existing command family:

- use `action: "plan"`;
- represent extension mode through an existing compatible data field, such as `data.operation: "extend"`;
- do not introduce `action: "plan.extend"` unless the repository already has a formally versioned dotted-action convention.

Exact fields must follow existing `CommandResult` contracts and deterministic ordering conventions.

15. Error Handling and Exit Codes

Use existing exit-code conventions:

Condition	Status	Exit code
Valid extension applied	passed	0
Valid extension preview	passed	0
Ordinary plan on a non-empty graph	blocked	2
No existing graph to extend	blocked	2

Condition	Status	Exit code
Placeholder is <code>in_progress</code> , <code>needs_review</code> , <code>done</code> , <code>replanned</code> , or <code>cancelled</code>	blocked	2
Placeholder has checkpoint or active packet	blocked	2
Placeholder ID not found	failed	3
Missing <code>--replace-placeholder</code> with <code>--extend</code>	failed	3
<code>--replace-placeholder</code> without <code>--extend</code>	failed	3
Extension metadata missing/invalid	failed	3
Entry or exit key missing/unknown	failed	3
Duplicate canonical/client key	failed	3
Missing dependency target	failed	3
Self-dependency or cycle	failed	3
Invalid state/schema	failed	3
Atomic persistence failure	failed	3
Neither stdin nor file input on import path	needs_input	10

The taxonomy remains:

Exit 2 = command is structurally valid but blocked by the current workflow position or target state
Exit 3 = invalid CLI combination, invalid structured input, invalid reference, or invalid canonical graph/state
Exit 10 = required input source is absent on an import path that can request input

Specific decisions:

- ordinary `aiqt plan` on a non-empty graph preserves `blockedOnState` behavior and exit 2;
- `aiqt plan --extend` on an empty graph returns exit 2, code `PLAN-EXTEND-GRAPH-EMPTY`, and recommends ordinary `aiqt plan`;
- `--extend` without `--replace-placeholder` returns exit 3, because the CLI option combination is structurally incomplete rather than interactively missing domain input.

Recommended stable error codes:

- `PLAN-EXTEND-GRAPH-EMPTY`;
- `PLAN-EXTEND-PLACEHOLDER-REQUIRED`;
- `PLAN-EXTEND-PLACEHOLDER-NOT-FOUND`;
- `PLAN-EXTEND-PLACEHOLDER-INELIGIBLE`;
- `PLAN-EXTEND-PLACEHOLDER-HAS-HISTORY`;
- `PLAN-EXTEND-ENTRY-INVALID`;
- `PLAN-EXTEND-EXIT-INVALID`;
- `PLAN-EXTEND-DUPLICATE-KEY`;
- `PLAN-EXTEND-DEPENDENCY-INVALID`;
- `PLAN-EXTEND-CYCLE`;
- `PLAN-EXTEND-PERSISTENCE-FAILED`.

Do not change the current `PLAN-GRAPH-NOT-EMPTY` behavior for ordinary non-extension planning: it remains a blocked workflow-state result with exit `2`.

16. Prompt and Agent-Driver Updates

Update planning prompts so agents understand progressive planning.

Required guidance:

- initial planning may keep future cuts as explicit roadmap placeholders;
- after a cut is completed, use `aiqt plan --extend` rather than editing canonical files;
- detail only the next cut unless the user asks for deeper planning;
- identify the placeholder work-unit ID;
- provide bounded new milestones/work units;
- declare entry and exit work units;
- preview before mutation;
- run graph validation/review/status after mutation;
- do not execute the first new work unit in the same planning-only session unless instructed.

`aiqt prompt driver` should mention extension when a non-empty graph contains a ready/planned roadmap placeholder and prior work is complete.

17. Repository Implementation Scope

Likely implementation areas:

Area	Expected change
<code>src/cli/commands/plan.command.ts</code>	Add <code>--extend</code> , <code>--replace-placeholder</code> , and extension-only <code>--preview</code> .
<code>src/cli/commands/import.command.ts</code>	Pass extension flags through stdin/file import plan delegation.
<code>src/services/planning-service.ts</code>	Add candidate-state extension path while preserving ordinary initial planning.
<code>src/services/graph-*</code> / dependency services	Validate and generate <code>blocks</code> and <code>requires</code> boundary dependencies across the full graph while preserving type.
<code>src/core/state</code> / repository writer	Reuse atomic write and derived-state recomputation.
<code>src/schemas/plan*</code>	Add extension metadata schema with backward-compatible parsing.
<code>src/schemas/state*</code>	Add minimal optional replacement linkage only if existing replanning metadata is insufficient.
<code>src/core/runlog</code>	Add deterministic extension/replanning event payloads using current conventions.
<code>src/services/prompt-service.ts</code>	Add progressive planning and extension guidance.
<code>src/templates/prompts/*</code>	Add extension examples for file/stdin paths.
<code>src/services/status/manage/review/export</code>	Confirm replanned placeholder and added graph render consistently; avoid duplicate logic.
<code>tests/unit</code>	Focused schema, eligibility, graph, dependency, candidate-state, and output tests.

Area	Expected change
tests/integration	End-to-end CLI extension, preview, import delegation, and Finan�as regression fixture.

Implementation may use different filenames where the repository has stronger current boundaries. Keep the change localized and use existing centralized graph/state services.

18. Runtime and Package Impact

- No new runtime package dependency is expected.
- No network access is required.
- No external service is required.
- No new canonical file is introduced.
- Preview is read-only.
- Existing project files outside `.aiqt/` are not modified.
- Extension input files remain optional/non-canonical.

19. Testing Scope

M17 must preserve the 878-test baseline and add focused tests.

19.1 Unit tests

Cover:

- extension schema valid/invalid cases;
- entry and exit key validation;
- placeholder eligibility by status;
- placeholder checkpoint/packet rejection;
- deterministic ID allocation;
- duplicate client-key and ID protection;
- incoming `blocks` dependency copying;
- incoming `requires` dependency copying with type preservation;
- outgoing `blocks` dependency copying;
- outgoing `requires` dependency copying with type preservation;
- mixed boundary dependency readiness;
- non-blocking `relates_to` behavior;
- original dependency preservation;
- cycle detection over old + new graph;
- `replanned` mutation and replacement linkage;
- current workflow recomputation;
- next recommended command;
- deterministic JSON output;
- no mutation during preview;
- no mutation during any failure.

19.2 Integration tests

Cover:

- file-input extension success;
- stdin/import extension success;
- preview through direct plan path;
- preview through import path;
- ordinary plan still returns `PLAN-GRAPH-NOT-EMPTY` with exit `2`;
- extension on an empty graph returns exit `2` and recommends ordinary planning;
- completed work units remain unchanged;
- exports/status/manage/review render the extended graph;
- `aiqt next --preview` selects the first new entry unit;
- downstream roadmap placeholder remains blocked until replacement exit units are done.

19.3 Atomicity tests

Simulate failure during persistence and confirm:

- no state mutation;
- no runlog append;
- no partial dependency addition;
- placeholder remains in original status;
- subsequent valid extension still receives correct IDs.

19.4 Finançes dogfood regression fixture

Create a fixture matching the observed state:

- 12 milestones;
- 21 work units;
- WU001-WU014 `done`;
- WU015 `ready` as Corte 1 placeholder;
- later cut placeholders planned/blocked;
- existing incoming and outgoing blocking dependencies around WU015.

Apply a detailed Corte 1 extension and verify:

- WU001-WU014 remain done and unchanged;
- WU015 becomes replanned;
- detailed Corte 1 milestones/work units are added;
- first detailed entry work unit is ready;
- later detailed work units are planned/blocked;
- the Corte 2 placeholder does not become ready prematurely;
- no manual canonical edit is required;
- full review/graph validation passes with only intentional roadmap warnings.

19.5 Mixed boundary-dependency fixture

Create a focused fixture in which one placeholder has all of the following simultaneously:

- incoming `blocks`;
- incoming `requires`;
- outgoing `blocks`;
- outgoing `requires`;

- one or more `relates_to` relationships.

Verify that:

- all incoming blocking dependencies are copied to every entry;
- all outgoing blocking dependencies are copied from every exit;
- each generated dependency preserves its original type;
- `relates_to` is not copied automatically and does not affect readiness;
- downstream readiness remains blocked until all generated blocking dependencies are satisfied.

19.6 Regression commands

Required:

```
pnpm typecheck
pnpm lint
pnpm test
pnpm build
pnpm validate
```

All existing and new tests must pass.

20. Definition of Done

M17 is complete when:

- `aiqt plan --extend` works against a non-empty graph;
- `aiqt import plan --stdin --extend` delegates to the same extension engine;
- `--replace-placeholder` targets exactly one eligible work unit;
- `--preview` validates and reports the extension without mutation;
- ordinary `aiqt plan` still refuses a non-empty graph without `--extend`;
- the placeholder is preserved and marked `replanned`;
- no new `superseded` status is introduced;
- existing completed records remain unchanged;
- incoming and outgoing `blocks` and `requires` dependencies are copied correctly with exact type preservation;
- original dependency records remain present;
- downstream work cannot become ready prematurely;
- duplicate keys, invalid references, and cycles are rejected;
- the complete candidate state is validated before persistence;
- writes are atomic;
- failed/preview operations do not append runlog events;
- successful extension appends deterministic extension/replanning history;
- status, manage, review, graph validate, next, checkpoint, and export continue to work;
- prompt templates explain progressive planning and safe extension;
- the Finanças fixture passes end-to-end;
- all prior 878 tests still pass;
- all M17 tests pass;
- `pnpm typecheck`, `pnpm lint`, `pnpm test`, `pnpm build`, and `pnpm validate` pass;
- no consumer project is manually edited by the M17 implementation.

21. Source-Control and Release Contract

Branch

```
feat/m17-incremental-work-graph-expansion
```

Primary commit

```
feat: add incremental work graph expansion
```

The detailed commit body must include:

- commands added;
- schema impact;
- mutation and atomicity behavior;
- dependency-boundary behavior;
- tests executed;
- known limitations;
- risk score from 0 to 100.

Milestone tag

```
m17-incremental-work-graph-expansion
```

Preserve the feature branch after merge according to current project practice.

22. Agent Execution Packet

Implement M17 on top of merged M16.

Starting point

- branch: `main`;
- merge commit: `d5dcb395f3ca56d36b38540960239403eef09f25`;
- tag: `m16-project-init-ux-root-configuration`;
- expected baseline: 878/878 tests passing.

Required implementation

1. Create branch `feat/m17-incremental-work-graph-expansion`.
2. Confirm the baseline tests and inspect the current plan/import/replan/state/dependency implementations.
3. Add extension flags to the direct plan and import-plan command paths.
4. Define the smallest backward-compatible extension input schema.
5. Implement placeholder eligibility checks.
6. Reuse the existing `replanned` status and metadata.
7. Build a candidate-state extension path.

8. Add entry/exit boundary copying for both `blocks` and `requires`, preserving each original type and without deleting old dependency records.
9. Validate the complete candidate graph before persistence.
10. Add read-only preview.
11. Persist atomically and append deterministic runlog history only after success.
12. Recompute statuses, workflow position, and next recommendation through existing centralized services.
13. Update prompt guidance for progressive planning.
14. Add the full unit, integration, atomicity, and Finanças dogfood test matrix.
15. Run all validation commands.
16. Create the detailed commit and milestone tag.
17. Do not modify the `financas-pessoais` project as part of M17 implementation.

Explicitly out of scope

- no direct `.aiqt/state.json` editing workaround;
- no second AIQT project for Corte 1;
- no arbitrary graph editor;
- no checkpoint amendment;
- no replacement of started/completed work;
- no automatic agent execution;
- no package publication unless separately authorized.

Appendix A — Minimum Test Matrix

ID	Scenario	Expected
A1	<code>aiqt plan --extend</code> on valid existing graph	Exit 0; extension applied
A2	Direct extension preview	Exit 0; no mutation/runlog
A3	Stdin/import extension	Exit 0; same canonical result as direct path
A4	Stdin/import extension preview	Exit 0; no mutation/runlog
A5	Ordinary plan on non-empty graph	Exit 2; <code>PLAN-GRAPH-NOT-EMPTY</code> unchanged
A6	Extend when graph is empty	Exit 2; <code>PLAN-EXTEND-GRAPH-EMPTY</code> ; recommend ordinary plan; no mutation
A7	Missing placeholder flag with <code>--extend</code>	Exit 3; no mutation
A8	Unknown placeholder	Exit 3; no mutation
A9	Placeholder <code>planned</code>	Allowed
A10	Placeholder <code>ready</code>	Allowed
A11	Placeholder blocked by dependencies	Allowed
A12	Placeholder <code>in_progress</code>	Exit 2; no mutation
A13	Placeholder <code>needs_review</code>	Exit 2; no mutation

ID	Scenario	Expected
A14	Placeholder <code>done</code>	Exit 2; no mutation
A15	Placeholder already <code>replanned</code>	Exit 2; no mutation
A16	Placeholder <code>cancelled</code>	Exit 2; no mutation
A17	Placeholder has checkpoint	Exit 2; no mutation
A18	Placeholder has active packet	Exit 2; no mutation
A19	Missing entry key	Exit 3; no mutation
A20	Missing exit key	Exit 3; no mutation
A21	Entry key not in new payload	Exit 3; no mutation
A22	Exit key not in new payload	Exit 3; no mutation
A23	Duplicate milestone client key	Exit 3; no mutation
A24	Duplicate work-unit client key	Exit 3; no mutation
A25	Dependency references missing unit	Exit 3; no mutation
A26	Self-dependency	Exit 3; no mutation
A27	Cycle across existing and new graph	Exit 3; no mutation
A28	Incoming <code>blocks</code> copied to entries	New canonical <code>blocks</code> dependencies created
A29	Incoming <code>requires</code> copied to entries	New canonical <code>requires</code> dependencies created; type preserved
A30	Outgoing <code>blocks</code> copied from exits	Downstream remains blocked by generated <code>blocks</code> dependencies
A31	Outgoing <code>requires</code> copied from exits	Downstream remains blocked by generated <code>requires</code> dependencies; type preserved
A32	Mixed incoming/outgoing <code>blocks</code> and <code>requires</code>	Readiness follows all original blocking semantics
A33	Existing <code>relates_to</code> relationships	Not copied automatically; no readiness effect
A34	Original dependencies preserved	No dependency deletion
A35	Placeholder becomes <code>replanned</code>	Preserved in state; never selected again
A36	Completed work units preserved	No semantic changes
A37	First replacement entry ready	Correct readiness recomputation
A38	Multiple exits	Downstream waits for all exits
A39	Persistence failure	Full rollback; no runlog append
A40	Successful extension runlog	Deterministic event(s) appended
A41	Status after extension	Correct current position/recommendation
A42	Review/manage/export after extension	Consistent rendering/classification
A43	<code>aiqt next --preview</code> after extension	Selects first new detailed WU
A44	Mixed boundary fixture with all dependency types	Correct copying, preservation, and readiness behavior
A45	Finanças fixture	Full end-to-end pass

ID	Scenario	Expected
A46	JSON command result compatibility	action remains plan ; operation reports extend in compatible data
A47	All previous tests	878/878 baseline preserved
A48	Full validation suite	typecheck/lint/test/build/validate pass

Appendix B — Finanças Acceptance Scenario

Initial state:

```
Project: financas-pessoais
Corte 0: completed
WU001-WU014: done
WU015: ready, roadmap placeholder for Corte 1
Corte 2+ placeholders: planned/blocked
Current work unit: none
```

Preview:

```
aiqt plan --extend \
  --replace-placeholder WU015 \
  --from-file .aiqt/exports/corte-1-plan.json \
  --preview
```

Preview must report:

```
Placeholder: WU015
Final placeholder status: replanned
Completed work units modified: 0
Completed milestones modified: 0
New milestones: N
New work units: N
New dependencies: N
Cycles introduced: 0
Mutation: false
```

Apply:

```
aiqt plan --extend \
  --replace-placeholder WU015 \
  --from-file .aiqt/exports/corte-1-plan.json
```

Postconditions:

```
WU001-WU014: done
WU015: replanned
First Corte 1 detailed entry: ready
Remaining Corte 1 detailed work: planned/blocked
Corte 2 placeholder: blocked until Corte 1 exit work completes
Manual state edits: none
```

Validation:

```
aiqt graph validate --json  
aiqt review  
aiqt status --json  
aiqt next --preview --json
```

Appendix C — Compatibility Rules

1. M17 is additive.
2. Existing projects load without migration work by the user.
3. Existing plan files remain valid for initial planning.
4. Extension metadata is required only with `--extend`.
5. Existing `PLAN-GRAPH-NOT-EMPTY` remains the normal-plan guard and returns exit `2`.
6. Extending an empty graph is a workflow-position block and returns exit `2`.
7. Invalid CLI combinations and invalid structured graph input return exit `3`.
8. Existing `replanned` behavior remains authoritative.
9. Existing centralized ID, dependency, status, recommendation, state-write, and runlog services must be reused.
10. `blocks` and `requires` remain distinct blocking dependency types and must preserve their canonical type during rewiring.
11. No canonical Markdown file is introduced.
12. AIQT remains a deterministic local workflow engine, not an AI runner.